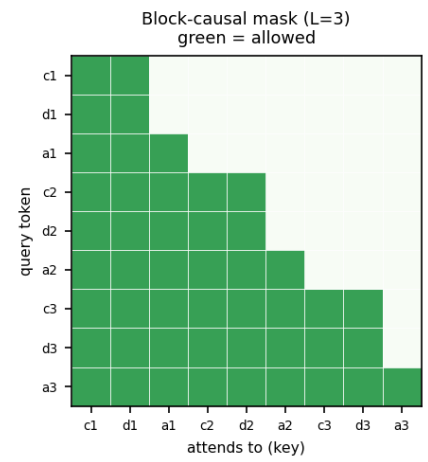




3. Data & label generation

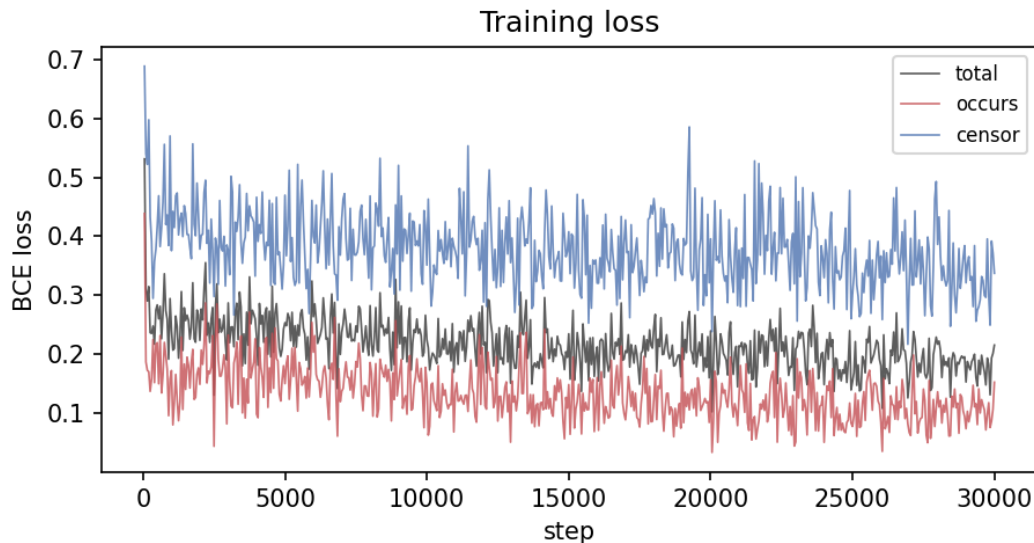
Cohort: MIMIC-IV in MEDS form, the v0.3.0 tensorized cohort (meds - torch - data tokenization; 11,958-code vocabulary; 292 train / 37 tuning / 37 held-out shards). Query sequences are sampled per patient context: a valid context is any event time at which the subject has ≥ 10 prior events. Each context gets the censor query plus 1–4 i.i.d. code queries (codes uniform over the full vocabulary, horizons log-uniform over 1–365 days), in random order.

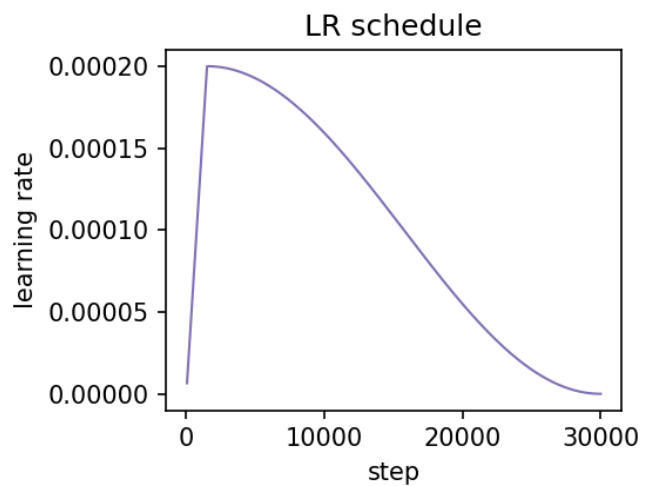
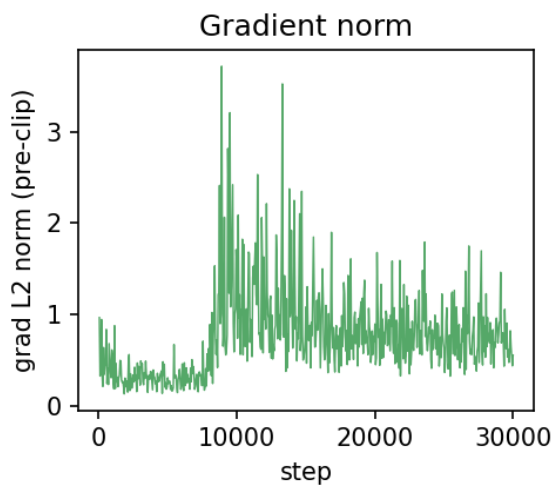
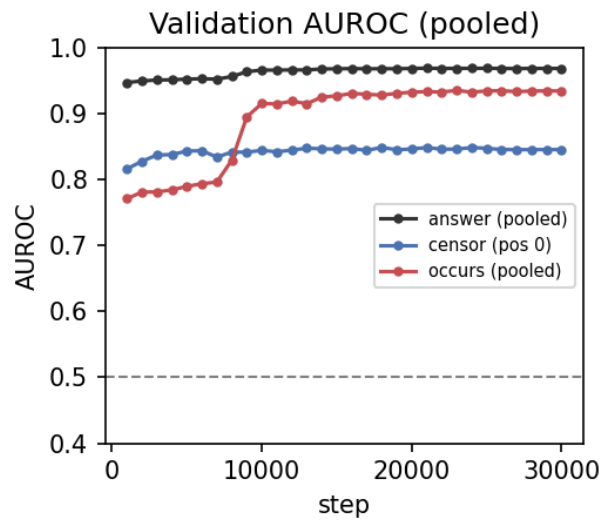
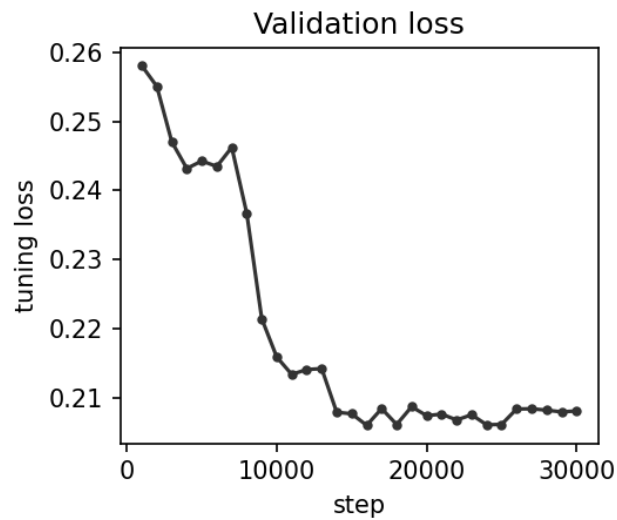
Labeling. A single backward `join_asof` per shard determines, for each (context, code, horizon), whether the code occurs in the window. Label semantics are three-valued: **True** if an event is observed in-window (an observed occurrence counts even if the record ends inside the window — essential for terminal events such as death); **False** if no in-window event and the window is fully observed; **null** (censored) if no in-window event and the window extends past the subject's last record. The censor query's answer is True iff the window end is at or before the last record.

split	sequences	query positions
train	598,016	—
tuning	37,888	—
held-out	37,888	132,567

4. Training & stability

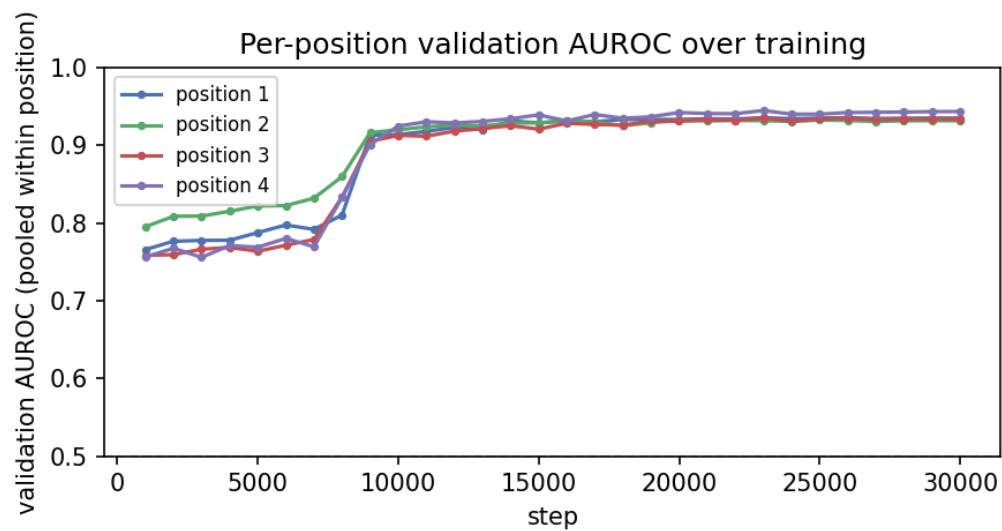
Optimizer AdamW (lr $2e-4$, weight decay 0.05, betas 0.9/0.98), cosine schedule with 5% warmup, gradient clipping at 1.0, bf16 mixed precision, batch size 96, 29999 steps on a single GB10 (~14k steps/h). Training was stable throughout: the gradient norm stayed bounded (median 0.68, max 3.71) with no loss spikes or divergence, and both validation heads improved monotonically before plateauing.





metric	value
best tuning loss	0.206
final tuning loss	0.208
final tuning censor AUROC (pos 0)	0.846
final tuning occurs AUROC (pooled)	0.935
median / max grad norm	0.68 / 3.71

Note these training-time AUROCs are pooled (computed across all codes at once) and so are base-rate inflated; the held-out within-query metrics in §5 are the trustworthy ones. They are shown here only to document training dynamics and stability.



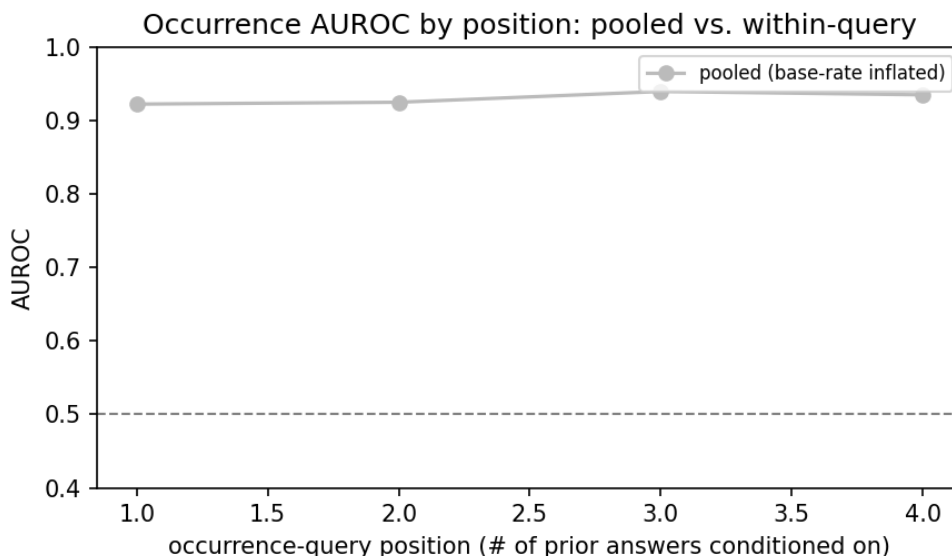
5. Evaluation I — random queries

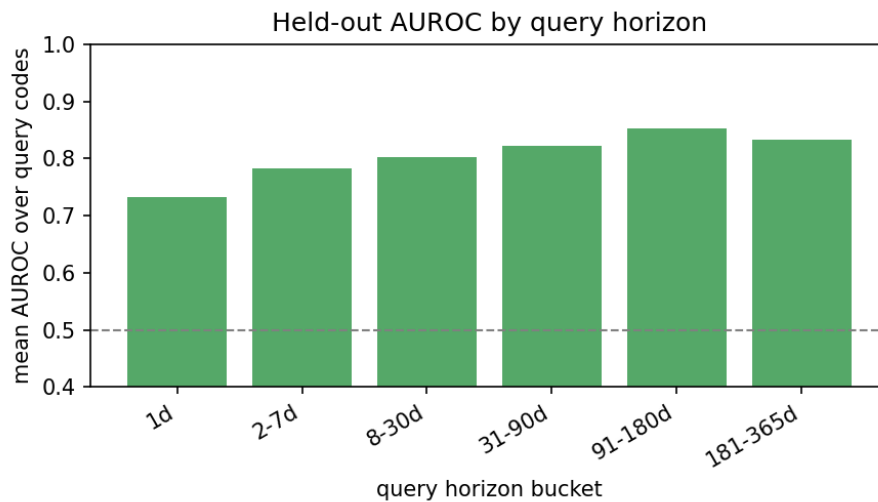
Held-out evaluation on randomly sampled query sequences (the same distribution as training, teacher-forced). Censored answers are dropped before AUROC.

Pooled vs. within-query AUROC. A naïve AUROC that pools all occurrence queries together is *base-rate inflated*: most positive/negative pairs it scores are *cross-query* (a positive for a common code vs. a negative for a rare one), which are separable just from the model learning per-code prevalence — not from answering any single query well. With codes drawn uniformly over a ~12k vocabulary spanning orders-of-magnitude prevalence, that inflation is large. The honest metric is **within-query AUROC** — AUROC computed inside each query code (so every scored pair shares a base rate) and then macro-averaged over codes. On this *random* held-out set, however, uniform sampling over ~12k codes leaves too few positives per code to form a stable within-query macro (no code clears the ≥ 10 -positive/ ≥ 10 -negative bar) — so the trustworthy within-query numbers come from the matched-code probe below and the clinical tasks (§6), both single-code-many-patients by construction. The pooled value is shown only to make the inflation explicit.

quantity	value
held-out sequences	37,888
sensor-query AUROC (position 0)	0.850
sensor-query prevalence	0.769
occurrence AUROC — pooled (base-rate inflated)	0.927
occurrence AUROC — macro within-query	—
(query codes with ≥ 10 pos & ≥ 10 neg)	0
occurrence prevalence	0.0513
occurrence censored fraction	0.218

Per-position (within-query) is the cleanest conditioning test. Occurrence queries are sampled i.i.d. at every position ≥ 1 , so positions differ only in how many prior teacher-forced answers they condition on. Macro-averaging the per-(query, position) AUROCs within each position therefore isolates the conditioning effect; a model that exploits the autoregressive context should hold or improve with position, never fall systematically.

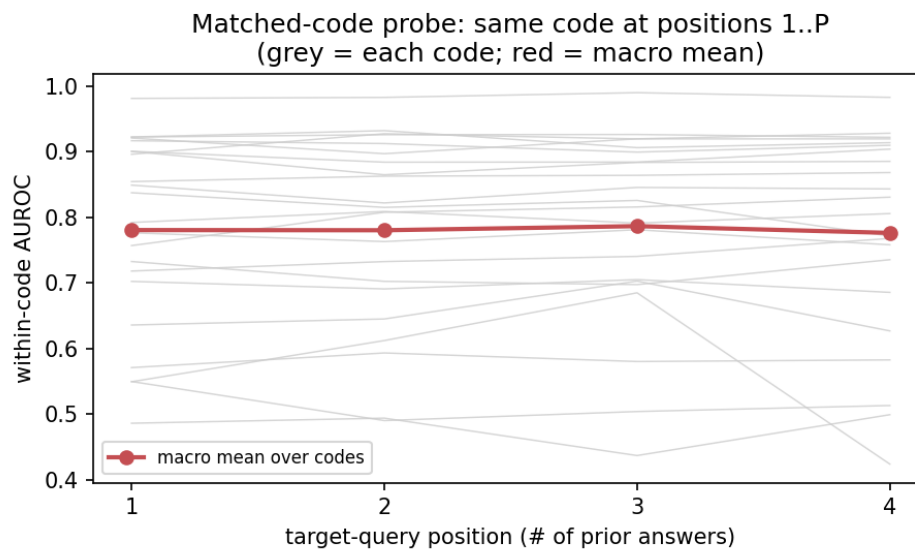




Matched-code position probe

The random-query per-position macro is limited by how often each code lands at each position. The probe removes that limitation: for ~20 curated common codes (admissions, discharges, ICU transfers, death, frequent labs / meds / diagnoses) the *same* code is placed at positions 1–4 across many held-out patients, with random filler queries before it. Per-(code, position) within-code AUROC then has ample positives, and tracking a fixed code across positions varies *only* the number of prior teacher-forced answers it conditions on — the definitive conditioning curve.

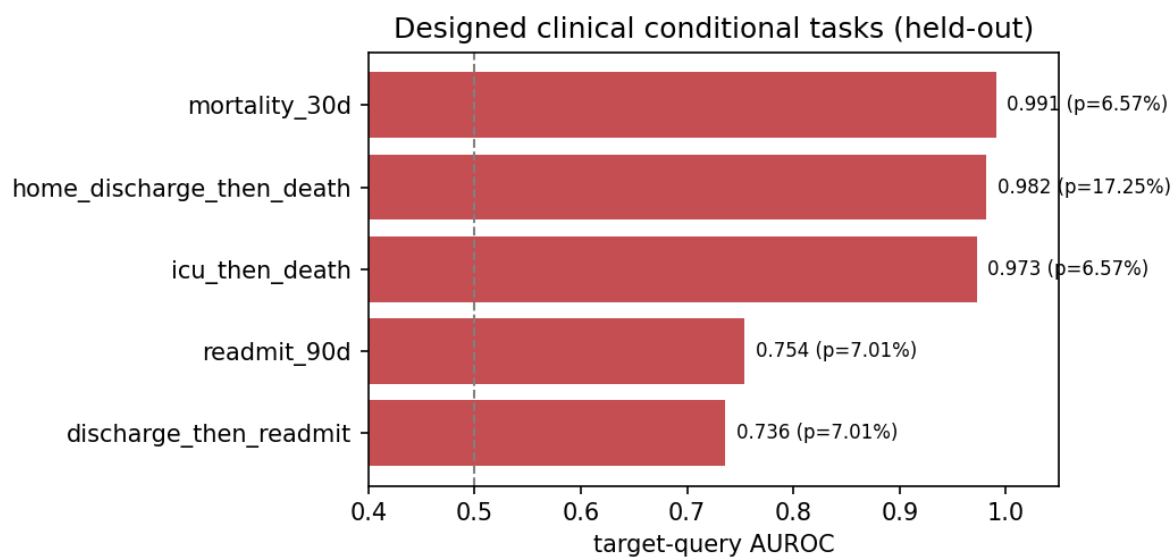
target position	# prior answers	macro within-code AUROC	# codes
1	0	0.781	22
2	1	0.780	22
3	2	0.787	22
4	3	0.776	22



6. Evaluation II — designed clinical conditional tasks

We hand-designed five clinically meaningful conditional query sequences, anchored 24 hours after a hospital admission on held-out subjects. Each begins with the censor query and ends with a target query; the intermediate queries set up the clinical condition. Because answers are teacher-forced, the target prediction is genuinely conditioned on the (observed) earlier outcomes.

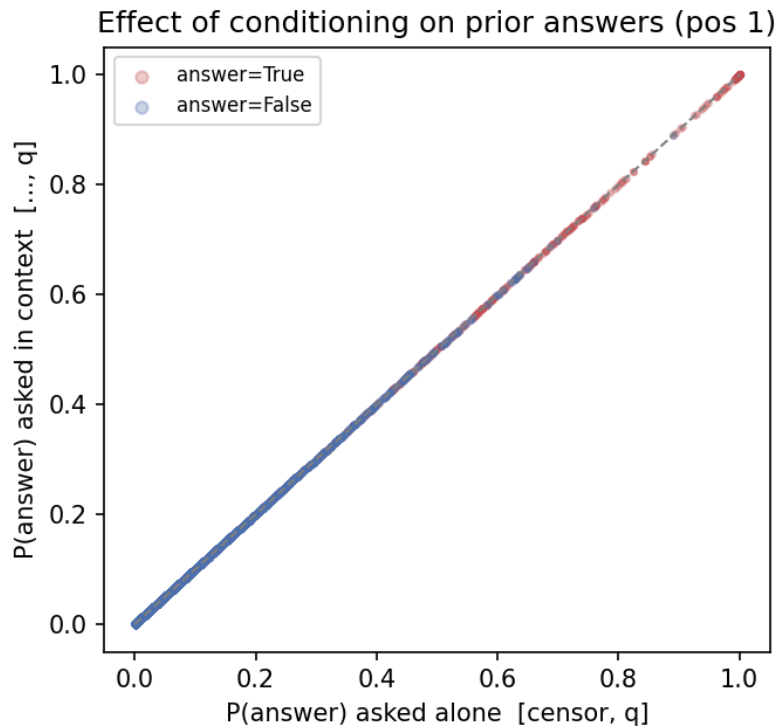
task	query sequence	n	prev.	target AUROC
discharge_then_readmit	[censor 90d] → home discharge 14d → ER readmit 90d	11,664	0.070	0.736
home_discharge_then_death	[censor 180d] → home discharge 30d → death 180d	11,807	0.173	0.982
icu_then_death	[censor 30d] → MICU admit 7d → death 30d	14,586	0.066	0.973
mortality_30d	[censor 30d] → death within 30d	14,586	0.066	0.991
readmit_90d	[censor 90d] → ER readmission within 90d	11,664	0.070	0.754



7. Evaluation III — does conditioning matter?

To confirm the model actually *uses* the conditional structure, we compare each occurrence query's prediction in two settings on matched (subject, time, code, horizon) tuples: asked **alone** as [censor, q], versus asked **in context** at its original position after preceding queries and their teacher-forced answers. A model that ignored prior answers would give identical predictions; a genuine conditional model shifts them.

pos	matched pairs	macro AUROC alone	macro AUROC in-ctx	mean ΔP	corr
1	15,674	—	—	0.000	1.000
2	15,762	—	—	0.008	0.966



8. Discussion & conclusions

The reformulation works. A single model, trained only on randomly assembled query sequences, answers held-out queries well above chance. On the matched-code probe — the cleanest within-query measurement, 22 common codes each scored against patients queried for the same code — macro AUROC is 0.776–0.787. The censor query reaches AUROC 0.850. (The pooled occurrence AUROC of 0.927 is reported only as a foil: it is base-rate inflated by cross-query comparisons, and the uniform 12k-code sampling leaves no single code with enough held-out positives to form a random-set within-query macro — which is precisely why the dense probe and the clinical tasks are the instruments we trust.)

Censoring is handled natively. Folding the censor question into the first query block removes the need for the separate censor head of the original EveryQuery while keeping every later query answerable: positions ≥ 1 are predicted even when their own answers are unobserved, because the always-observed censor answer and the block-autoregressive structure absorb the missingness.

Conditioning: present, and used when it is informative. As a sanity check, at position 1 the in-context and alone predictions are identical (mean $|\Delta P| = 0$, correlation 1.0) — there [censor, q] in context is the singleton, so the harness is consistent. At position 2, conditioning on one real prior answer shifts the prediction by mean $|\Delta P| = 0.008$ (correlation 0.966) — small, because in randomly assembled sequences the prior query is an arbitrary code whose answer is mostly uninformative about the target. The decoder *can* condition (the architecture is verified to, by unit tests, and the clinical chains below exploit it strongly); it simply does not manufacture dependence where the data carry none. The matched-code probe makes the same point structurally — holding a code fixed and adding more random prior answers (positions 1→4) leaves within-code AUROC essentially flat (0.776–0.787), because random priors carry no signal about the target. Where the prior *is* informative, the model uses it strongly: on designed clinical sequences it reaches AUROC 0.991 (mortality 30d), e.g. 30-day mortality and ICU-then-death chains — the capability the block-autoregressive form was built to add. The unit tests independently verify the mechanism: flipping a prior answer changes later predictions, while a query's own answer never leaks into its own prediction.

On metrics. Pooled AUROC across heterogeneous query codes is base-rate inflated — most scored pairs are cross-query and separable by prevalence alone — so the trustworthy numbers here are within-query: the

matched-code probe and the clinical tasks are single-code-many-patients by construction. The random-set per-query macro is undefined here only because uniform sampling over ~12k codes yields too few positives per code on held-out; a denser code sampling would recover it.

Limitations. Evaluation is teacher-forced (earlier answers are ground truth, not model samples), so it measures conditional calibration, not free-running rollout. The conditioning effect on random sequences is small by construction (uninformative random priors); a probe with deliberately correlated query pairs would size the effect when priors are predictive. Sequences are capped at five queries and assembled in random order, by design — natural temporal ordering and longer chains are natural next steps.

Code: forked from [payalchandak/EveryQuery](#); conditional pipeline on branch `conditional-queries`. New CLIs: `EQ_generate_query_sequences`, `EQ_predict_sequences`, `EQ_evaluate_sequences`. All unit + E2E tests pass.